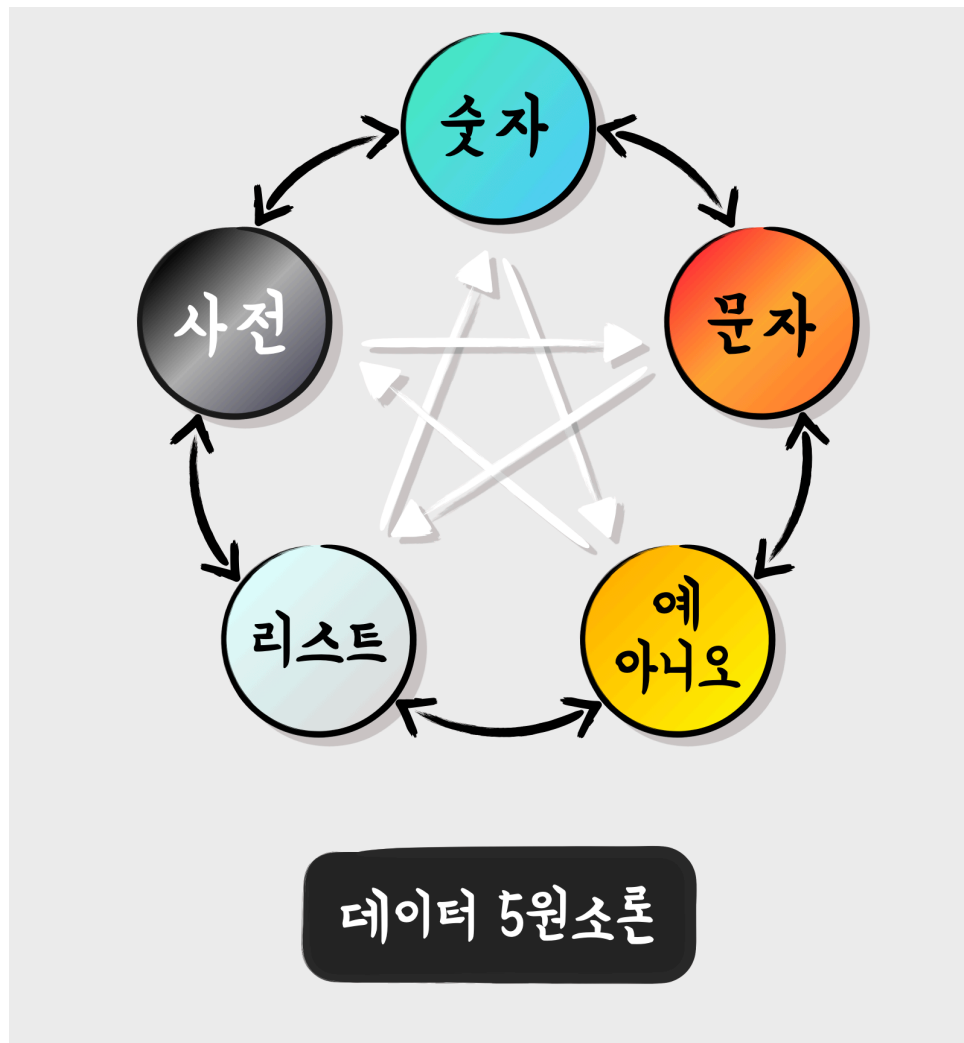




Python 기초 문법 Part3



1. 딕셔너리
2. 튜플
3. 세트
4. 데이터 타입 마무리



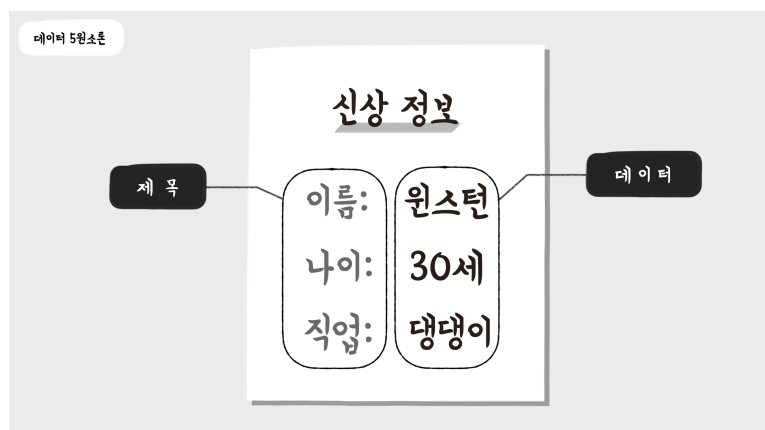
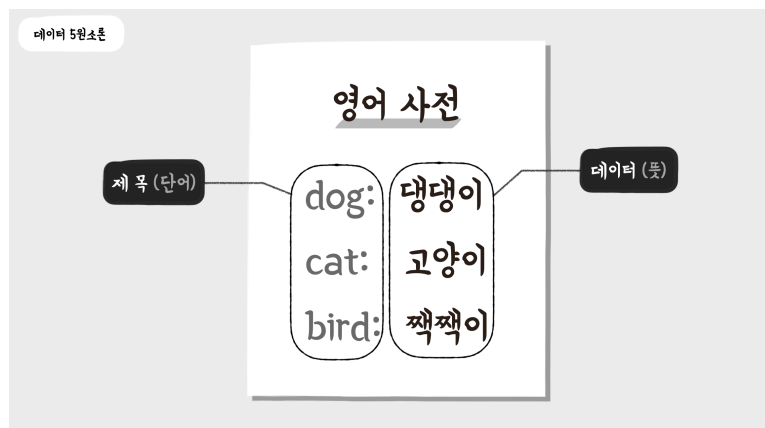
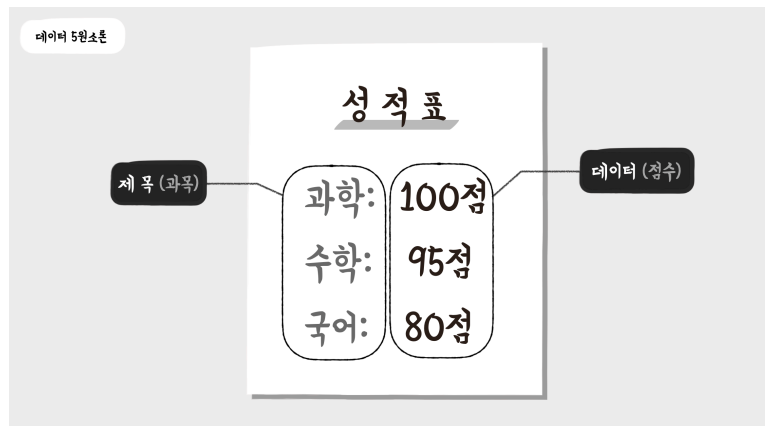
▼ 1. 딕셔너리





딕셔너리는 **값(Value)에 이름(Key)를 붙여서** 관리하는 자료구조입니다.
 연락처 앱에서 이름을 찾으면 연락처가 나오는 것처럼,
 이름만 알고 있으면 원하는 값을 빠르게 찾을 수 있습니다 📖

▼ 딕셔너리란?

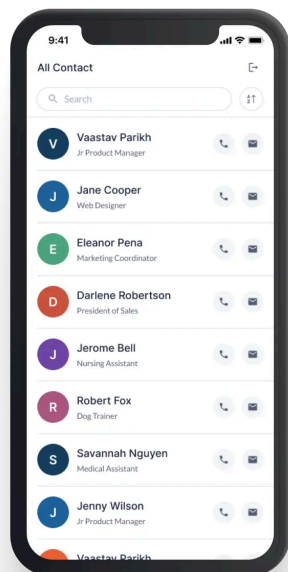


딕셔너리명 = { 이름1: 값1, 이름2: 값2, 이름3: 값3 }

```
score = {  
    "김기범": 100,  
    "김바둑": 95,  
    "김개똥": 100,  
}
```

```
empty_dict = {}
```

- 딕셔너리는 여러 개의 데이터에 각각 아이디를 붙여서 관리하는 자료구조입니다.
- 데이터에 아이디만 붙여놓으면, 아이디만 가지고 쉽게 값을 찾을 수 있습니다.
- **예시:** 전화번호부 (이름으로 전화번호 찾기), 사전 (단어로 뜻 찾기), 학생부 (학번으로 학생 정보 찾기)



▼ 리스트와의 차이?

- 둘 다 공통적으로 여러 데이터를 묶어서 관리하는 자료구조입니다.

- **리스트**: '순서'로 데이터를 구분하고 쉽게 찾을 수 있습니다.
- **딕셔너리**: '아이디'로 데이터를 구분하고 쉽게 찾을 수 있습니다. 리스트와 달리 순서가 없습니다.

▼ 딕셔너리의 '키' 이해하기



'키'란 값에 붙인 아이디를 의미합니다.

▼ 키(아이디)의 특징

1. 키와 값은 하나의 쌍으로 구성됩니다.
2. 키는 중복될 수 없습니다.
3. 키는 숫자나 문자 등으로 만들 수 있습니다.

▼ 특정 키의 값 만들기

딕셔너리[키] = 값

```
student = {} # 빈 딕셔너리 생성
student["name"] = "김바둑" # 새로운 키-값 쌍 추가
```

- 리스트의 인덱스에 값을 넣는 것처럼, 딕셔너리도 똑같이 사용하면 됩니다!

▼ 특정 키의 값 찾기

```
student["name"] # 키가 없으면 KeyError 발생

# 더 안전한 방법
student.get("age") # 키가 없으면 None 반환
student.get("phone", "없음") # 키가 없을 때 기본값 지정 가능
```

▼ 딕셔너리[키]

- **딕셔너리[키]** 방식은 해당 키가 없을 때 에러를 발생시키므로 주의가 필요합니다.

- 직관적이고 간단하지만, 키가 존재하지 않을 때 프로그램이 중단될 수 있습니다.

▼ get

- `get()` 메소드는 더 안전한 방식으로, 키가 없을 때도 에러 없이 `None` 이나 지정된 기본값을 반환합니다.
- 특히 기본값을 설정할 수 있어서 키가 없는 상황을 유연하게 처리할 수 있습니다.
 - `student.get("age", 0)` 처럼 사용하면, age 키가 없을 때 0을 반환합니다.

▼ 특정 이름의 값 수정하기

```
# 직접 기존의 값 수정
student["age"] = 21
```

작업	예시	설명
직접 수정	<code>student["age"] = 25</code>	• 가장 기본적인 방법 • 키가 없으면 새로 생성됨 • 키가 있으면 값이 덮어쓰워짐
update() 사용	<code>student.update({"age": 25})</code>	• 여러 키-값 쌍을 한번에 수정 가능 • 딕셔너리 형태로 전달

▼ 특정 이름의 값 삭제하기

```
del student["age"] # del 키워드 사용

grade = student.pop("grade") # 삭제된 값 반환

student.clear() # 모든 키-값 쌍 삭제
```

메소드	설명	사용 예시	유의 사항
<code>del</code>	특정 키-값 삭제	<code>del score["age"]</code>	• 반환값 없음 • 존재하지 않는 키 삭제 시 에러 발생 • 단순 삭제만 필요할 때 사용

<code>pop()</code>	특정 키-값 삭제 및 반환	<code>age = score.pop("age", 25)</code>	- 삭제된 값 반환 - default 값 지정 가능 - 삭제된 값을 활용해야 할 때 사용
<code>clear()</code>	모든 키-값 삭제	<code>score.clear()</code>	- 딕셔너리를 비움 - 빈 딕셔너리 상태가 됨 - 실행 취소 불가능

▼ 주요 딕셔너리 함수

함수	설명	사용법	유의 사항
<code>keys()</code>	모든 키를 받아올 수 있음	<code>dict.keys()</code>	리스트로 변환하려면 <code>list()</code> 로 감싸야 함
<code>values()</code>	모든 값을 받아올 수 있음	<code>dict.values()</code>	리스트로 변환하려면 <code>list()</code> 로 감싸야 함

```
menu = {
    "커피": 3000,
    "케이크": 5000
}
```

```
menu.keys()          # dict_keys(['커피', '케이크'])
menu.values()        # dict_values([3000, 5000])
```

```
list(menu.keys())    # ["커피", "케이크"]
list(menu.values())  # [3000, 5000]
```

▼ 실습

▼ 1. 동물원 문제

```
zoo = {
    "사자": 4,
    "호랑이": 3,
    "기린": 7,
    "코끼리": 5,
    "원숭이": 12,
```

```

"팬더": 2,
"캥거루": 8,
"코알라": 6,
"하마": 3,
"얼룩말": 9,
"펭귄": 15,
"악어": 4,
"뱀": 8,
"거북이": 12,
"앵무새": 10,
"물개": 7,
"바다사자": 5,
"수달": 4,
"재규어": 2,
"치타": 3,
"표범": 2,
"하이에나": 6,
"기니피그": 20,
"카멜레온": 6,
"이구아나": 4,
"고릴라": 3,
"침팬지": 5,
"오랑우탄": 4,
"기러기": 11,
"백조": 6,
"순록": 9,
"토끼": 25,
"비버": 4,
"낙타": 6
}

```

1. 동물원(**zoo**)에서 **펭귄** 은 총 몇 마리인가요?
2. 동물원에 **해달** 이 새로 들어왔습니다. 해달 5마리를 추가해보세요.
3. 동물원에 **토끼** 가 추가로 3마리가 더 들어왔습니다. 딕셔너리를 수정해보세요.
4. **이구아나** 가 다른 동물원으로 이전했습니다. 딕셔너리에서 삭제해보세요.

▼ 코드 예시

```
# Q1. 동물원(zoo)에서 펭귄은 총 몇 마리인가요?
zoo["펭귄"]

# Q2. 동물원에 해달이 새로 들어왔습니다. 해달 5마리를
# 추가해보세요.
zoo["해달"] = 5
print(zoo["해달"])

# Q3. 동물원에 토끼가 추가로 3마리가 더 들어왔습니다.
# 딕셔너리를 수정해보세요.
rabbit = zoo.get("토끼", 0)
zoo["토끼"] = rabbit + 3
print(zoo["토끼"])

# Q4. 이구아나가 다른 동물원으로 이전했습니다. 딕셔너리
# 에서 삭제해보세요.
del zoo["이구아나"]
```

▼ 2. 학생 성적 관리

```
students_score = {
    "아이언맨": {"수학": 85, "영어": 90, "과학": 95},
    "토르"      : {"수학": 92, "영어": 87, "과학": 70},
    "헐크"      : {"수학": 10, "영어": 80, "과학": 100}
}
```

위와 같은 학생 성적이 주어졌을 때, 다음 문제를 풀어보세요 :

1. 새로운 학생과 점수를 추가해보기. (ex. 캡틴 아메리카)

▼ 코드 예시

```
# 1. 새로운 학생과 점수를 추가해보기. (ex. 캡틴 아
# 메리카)
students_score["캡틴 아메리카"] = {"수학": 10,
    "영어": 100, "과학": 0}
students_score.update(
    {"캡틴 아메리카": {"수학": 10, "영어": 100,
```

```
"과학": 0}}  
)
```

2. 아이언맨이 성적에 이의제기 했습니다. 아이언맨의 **영어 점수를 95점**으로 높여주세요.

▼ 코드 예시

```
# 2. 아이언맨이 성적에 이의제기 했습니다. 아이언맨의  
영어 점수를 95점으로 높여주세요. ["영어"] = 95  
students_score["아이언맨"]["영어"] = 95
```

3. 전체 학생의 **수학 점수 평균**을 구해보세요.

▼ 힌트

```
# 학생의 수를 구하는 방법  
len(students_score.keys())
```

▼ 코드 예시

```
# 3. 전체 학생의 수학 점수 평균을 구해보세요  
  
scores = list(students_score.values())  
  
math_scores = []  
  
for score in scores:  
    math_scores.append(score["수학"])  
  
math_average = sum(math_scores) / len(math_scores)  
print(math_average)
```

4. 보너스:

수학 최고 점수를 찾아보세요.

▼ 코드 힌트: 딕셔너리의 `values()` 함수를 활용하면 값을 순회할 수 있습니다.

```

math_champion = -1

# 각 학생들의 점수들만 긁어옴
scores = list(students_score.values())

# 첫번째 score부터 마지막 score까지 아래 내용을 실행합니다.
for score in scores:
    # 각 score에 접근할 때마다 수학 점수를 긁어온다.
    math_score = score["수학"]

    # math_score를 활용해서 챔피언을 갱신시켜준다.
    if math_champion < math_score:
        math_champion = math_score

print(math_champion)

```

▼ 힌트

- 새로운 학생 추가할 때는 중첩 딕셔너리 구조를 고려하세요.
- 평균 계산시 `values()` 함수를 활용하면 편리합니다.
- 평균 계산시, `sum()` 함수와 `len()` 함수를 사용하면 편리합니다.
 - 숫자 리스트를 `sum()`으로 감싸주면 합계를 구할 수 있습니다.
 - 숫자 리스트를 `len()`으로 감싸주면 갯수를 구할 수 있습니다.
- 딕셔너리의 `items()` 함수를 활용하면 각각의 키-값을 순회할 수 있습니다.

▼ 2. 튜플



튜플은 여러 데이터를 순서대로 저장하는 자료구조입니다.
도시락 속 반찬처럼 한번 넣으면 바꿀 수 없게 고정되어 있어서,
중요한 데이터를 안전하게 보관할 때 사용합니다 📦 리스트와 딕셔너리의 차이

▼ 튜플이란?

튜플명 = (값1, 값2, 값3)

튜플명 = tuple(리스트)

직접 튜플 만들기

`coordinates = (37.5665, 126.9780)` # 서울의 위도, 경도

`rgb_color = (255, 128, 0)` # RGB 색상값

빈 튜플 만들기

`empty_tuple = ()`

리스트로부터 튜플 만들기

```
numbers = [1, 2, 3, 4, 5]
numbers_tuple = tuple(numbers) # (1, 2, 3, 4, 5)
```

- 데이터를 '고정'된 형태로 저장합니다. 실수로 데이터의 위치나 값이 변경되는 것을 방지할 수 있습니다.
- 튜플은 리스트와 비슷하지만, **한 번 만들면 수정할 수 없습니다.**

▼ 리스트와 튜플의 차이점

특징	리스트	튜플
수정 가능 여부	수정 가능	수정 불가능
선언 방법	[] 사용	() 사용
메모리 사용	더 많이 사용	더 적게 사용
실행 속도	상대적으로 느림	상대적으로 빠름



튜플이 메모리 사용과 속도가 더 빠른 이유?

리스트는 수정될 가능성이 있기 때문에 여분의 메모리도 확보해두고, 추가적인 검사와 처리가 필요합니다.

튜플은 수정될 가능성이 없기 때문에 리스트에 비해 빠르고 메모리를 적게 사용합니다.

▼ 튜플의 주요 기능

▼ 튜플 만들기

```
# 기본적인 튜플 생성
point = (3, 4)

# 여러 자료형을 담은 튜플
mixed = (1, "hello", 3.14)

# 튜플 패킹
coordinates = 37.5665, 126.9780 # 괄호 생략 가능
```

▼ 튜플 언패킹

변수1, 변수2, 변수3 = 튜플

```
name, age, height = ("김바둑", 25, 170)
print(name)      # "김바둑"
print(age)       # 25
print(height)    # 170

# 리스트처럼 n번째 위치에 있는 요소도 접근 가능!
coordinate = (30, 120)
coordinate[0] # 0번째 요소
coordinate[1] # 1번째 요소
```

- 튜플 언패킹

도시락을 열어서 반찬을 하나씩 꺼내는 것처럼,
튜플의 각 값을 개별 변수에 나누어 담는 것을 의미합니다 🍱

- 주의

튜플의 값 개수와 변수의 개수가 정확히 일치해야 합니다!

▼ 실습

▼ 1. 미니 실습

```
# 과일 목록 (과일, 색상)
fruits = [
    ("사과", "빨강"),
    ("바나나", "노랑"),
    ("포도", "보라"),
    ("오렌지", "주황"),
    ("키위", "초록"),
    ("체리", "빨강"),
    ("블루베리", "남색"),
    ("망고", "노랑"),
    ("수박", "초록"),
    ("딸기", "빨강"),
    ("자두", "보라"),
    ("복숭아", "분홍"),
    ("레몬", "노랑"),
```

```

("라임", "초록"),
("아보카도", "초록"),
("석류", "빨강"),
("용과", "분홍"),
("파파야", "주황"),
("두리안", "노랑"),
("코코넛", "갈색"),
("패션프루트", "보라"),
("리치", "빨강"),
("구아바", "초록"),
("자몽", "분홍")
]

```

1. 위 튜플의 리스트에서 각 **과일들의 색상들**을 하나씩 출력해보세요.

▼ 코드 예시

```

for fruit in fruits:
    # fruit는 (과일, 색상)으로 이루어져있는 튜플
    name, color = fruit
    print(color)

```

2. 위 튜플 리스트에서 '**주황**'색 과일들만 찾아서 **이름**을 출력해보세요.

▼ 코드 예시

```

for fruit in fruits:
    # fruit는 (과일, 색상)으로 이루어져있는 튜플
    name, color = fruit
    if color == "주황":
        print(name)

```

▼ 2. 좌표 시스템 만들기

아래와 같은 좌표들이 주어졌을 때,
원점 (0, 0) 으로부터 **가장 멀리 있는 점의 거리**를 찾아서 출력해보세요.

```

points = {
    "A": (3, 4),          # A점
    "B": (-1, 2),         # B점
}

```

```

"C": (5, -3),      # C점
"D": (2, -5),      # D점
"E": (7, 1),       # E점
"F": (-4, -2),     # F점
"G": (0, 6),       # G점
"H": (8, -4),      # H점
"I": (-3, 5),      # I점
"J": (4, 7),       # J점
"K": (-2, -6)      # K점
}

```

▼ 출력 예시

원점에서 가장 멀리있는 점까지의 거리는 ▲입니다.

▼ 코드 예시

```

from math import sqrt

# points에 있는 좌표값들만 짝 끊어와서 리스트로 만들어줌
coordinates = list(points.values())

# 현재 챔피언은 무주공산임
max_distance = -100000000 # (최약체)
min_distance = 100000000  # (최강체)

# coordinates를 0번부터 끝까지 순회하면서,
# 원점에서 가장 멀리 떨어진 점을 찾아보기
for coordinate in coordinates:
    x, y = coordinate # coordinate는 (x, y) 형태로 되어있는 튜플
    distance = sqrt(x ** 2 + y ** 2) # 원점과 coordinate 사이의 거리

    # 계산한 distance를 max_distance와 비교하면서 최대값을 갱신
    if max_distance < distance:
        max_distance = distance

```

```

# 계산한 distance를 min_distance와 비교하면서 최
소값을 갱신
if min_distance > distance:
    min_distance = distance

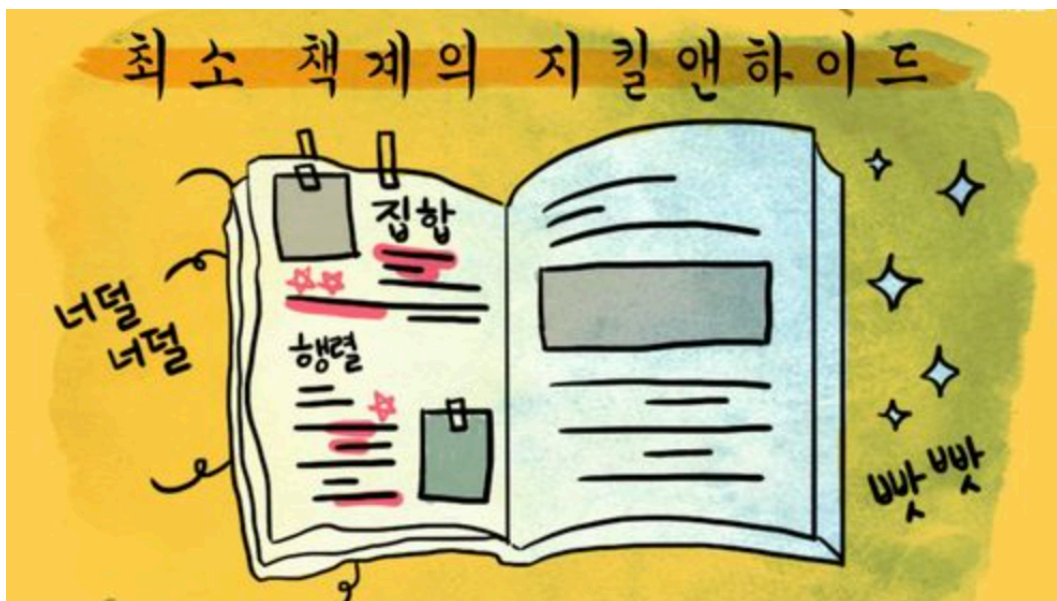
print(f"원점에서 가장 멀리있는 점까지의 거리는 {max_dis
tance} 입니다.")
print(f"원점에서 가장 가까있는 점까지의 거리는 {min_dis
tance} 입니다.")

```

▼ 힌트

- 튜플 언패킹을 활용하면 x, y 좌표를 쉽게 분리할 수 있습니다.
- 거리 계산 공식: $\sqrt{x^2 + y^2}$
- 제곱근을 계산하려면 `math` 모듈의 `sqrt()` 함수를 사용하세요.

▼ 3. 세트





세트(집합)는 데이터가 중복되지 않도록 관리해주는 자료구조입니다 🧺

▼ 세트란?

세트명 = { 값1, 값2, 값3 }

세트명 = set(리스트)

```
# 직접 세트 만드는 법
numbers = {1, 2, 3, 3, 3}           # 결과: {1, 2, 3}

# 리스트로부터 세트 만드는 법
fruits = ["사과", "바나나", "딸기", "사과"]
fruits_set = set(fruits)           # 결과: {"사과", "바나나", "딸기"}

# 빈 세트 만드는 법
empty_set = set()
```

- 데이터가 중복되지 않도록 자동으로 관리됩니다.
- 데이터의 순서가 없어서 인덱스로 접근할 수 없습니다.

▼ 세트의 주요 기능

▼ 집합 연산

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# 합집합
A | B           # {1, 2, 3, 4, 5, 6}
A.union(B)

# 교집합
A & B           # {3, 4}
```

```
A.intersection(B)
```

```
# 차집합
```

```
A - B          # {1, 2}
```

```
A.difference(B)
```

▼ 데이터 관리

```
# 요소 추가
```

```
fruits = {"사과", "바나나"}
```

```
fruits.add("딸기")
```

```
# 요소 제거
```

```
fruits.remove("바나나")
```

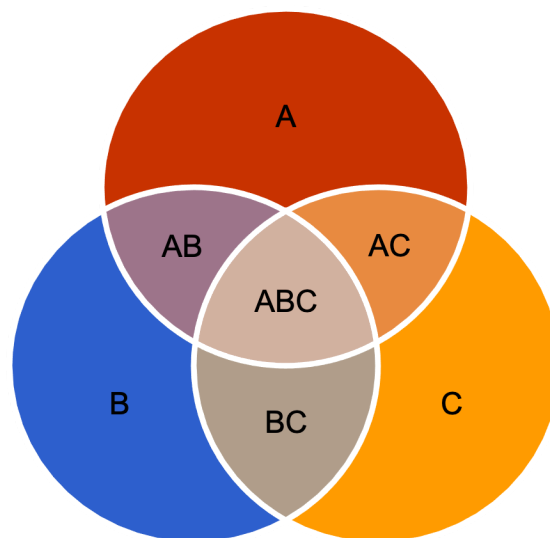
```
# 모든 요소 제거
```

```
fruits.clear()
```

▼ 실습

▼ 수강 그룹 관리하기

다음과 같은 수강 그룹이 있을 때, 아래 문제들을 풀어보세요:



```
python_class = {"철수", "영희", "민수", "길동", "영수"}
```

```
java_class = {"영희", "길동", "철민", "영수", "진희"}
```

```
web_class = {"민수", "길동", "수진", "영수", "진희"}
```

1. 모든 수업을 수강하는 학생들을 찾아보세요. (힌트: 교집합)
2. Python과 Java 수업은 듣지만, Web 수업은 듣지 않는 학생들을 찾아보세요. (힌트: 교집합과 차집합)
3. 적어도 두 개 이상의 수업을 듣는 학생들의 명단을 만들어보세요. (힌트: 교집합, 합집합)
4. 한 개의 수업만 듣는 학생들을 찾아보세요. (힌트: 합집합, 차집합)

▼ 코드 예시

```
# 1. 모든 수업을 수강하는 학생들
all_classes = python_class & java_class & web_class

# 2. Python과 Java 수업은 듣지만 Web 수업은 듣지 않는 학생들
python_and_java_only = (python_class & java_class) - web_class

# 3. 적어도 두 개의 수업을 듣는 학생들
two_or_more = (python_class & java_class) | (java_class & web_class) | (python_class & web_class)

# 4. 한 개의 수업만 듣는 학생들
one_class = (python_class | java_class | web_class) - two_or_more
```

▼ 4. 데이터 타입 마무리

▼ 맛집 정보 시스템

restaurants.py

```

restaurants = [
    {
        "name": "진진가연",
        "description": "홍대의 중화요리 전문점으로, 다양한
중식 메뉴를 제공합니다. 특히 해산물 육수를 사용한 짬뽕이 유명합
니다.",
        "category": "중식",
        "location": (37.557545, 126.924476),
        "rating": 3.0,
        "keywords": [
            "짬뽕",
            "해산물",
            "중식"
        ],
        "reviews": [
            "짬뽕 국물이 시원하고 해산물이 가득해요.",
            "가격 대비 맛도 훌륭합니다."
        ]
    },
    {
        "name": "자연별곡 홍대점",
        "description": "제철 식재료를 활용한 건강한 한식을
제공하는 뷔페식당입니다.",
        "category": "한식",
        "location": (37.556321, 126.922396),
        "rating": 3.3,
        "keywords": [
            "한식",
            "뷔페",
            "건강식"
        ],
        "reviews": [
            "다양한 한식을 뷔페식으로 즐길 수 있어요.",
            "재료가 신선하고 메뉴 구성이 알차요."
        ]
    },
    {
        "name": "우와",

```

```

        "description": "오코노미야키와 철판구이를 전문으로
하는 일식 퓨전 레스토랑입니다.",
        "category": "일식",
        "location": (37.557045, 126.922211),
        "rating": 3.5,
        "keywords": [
            "오코노미야키",
            "철판구이",
            "일식"
        ],
        "reviews": [
            "참마로 만든 오코노미야키가 아주 맛있어요.",
            "소스 조합이 정말 훌륭해요."
        ]
    },
]

```

▼ 데이터 읽어보기

- 주어진 데이터는 리스트 안에 딕셔너리(식당)들이 담겨 있습니다.
- 이 리스트는 `restaurants` 라는 변수에 담겨 있습니다.
- ▼ `restaurants` 의 첫번째 식당의 이름 정보를 추출하려면?

```
restaurants[0]["name"]
```

- ▼ `restaurants` 에 담긴 모든 식당들의 이름을 출력하려면?

```

for restaurant in restaurants:
    print(restaurant["name"])

```

- ▼ `restaurants` 에 담긴 모든 식당들의 평점을 출력하려면?

```

for restaurant in restaurants:
    print(restaurant.get("rating"))

```

```
for restaurant in restaurants:
```

```
if "rating" in restaurant:
    print(restaurant["rating"])
```

▼ 위 레스토랑 데이터를 활용해서 다음 정보를 추출해보세요.

1. 평점이 4점 이상인 식당의 이름들을 출력해서 보여주세요.

▼ 코드 예시

rating 정보가 restaurant에 있는지 먼저 꼼꼼하게 체크해줘야 합니다!

```
# 1번 방법
for restaurant in restaurants:
    if "rating" in restaurant:
        if restaurant["rating"] >= 4.0:
            print(restaurant["name"])

# 2번 방법
for restaurant in restaurants:
    if "rating" in restaurant and restaurant["rating"] >= 4.0:
        print(restaurant["name"])
```

2. **category** 가 **한식** 카테고리에 속하는 식당들을 출력해보세요.

▼ 코드 예시

딕셔너리에서 get 동사를 쓰면 좀 더 안전하게 값을 참조할 수 있습니다.

```
for restaurant in restaurants:
    if restaurant.get("category") == "한식":
        print(restaurant.get("name"))
```

3. **category** 가 **일식** 카테고리에 속하는 식당들의 **평균 평점**을 계산해보세요.

▼ 코드 예시

빈 소쿠리를 채워가는 전략을 써봅시다!

```
# 평균 평점
# 1. '일식'에 해당하는 식당들의 점수들을 '수집'한다.
```

```

# 2. 수집한 점수들의 합계와 갯수를 구한다.
# 3. 평균을 구한다.

japanese_ratings = []

for restaurant in restaurants:
    if restaurant.get("category") == "일식":
        # category 값이 만약에 일식이라면, rating
        # 값을 추출한다.
        # 혹시나 rating 값이 없는 경우에는, 0을 대
        # 입한다.
        rating = restaurant.get("rating", 0)
        japanese_ratings.append(rating)

average = sum(japanese_ratings) / len(japanese_ratings)
print(average)

```

4. 식당들의 `keywords` 로 어떤 종류의 키워드들이 있는지 정리하고자 합니다. 모든 식당들의 키워드들을 수집해서 하나의 집합으로 만들어보세요. (단 중복이 없어야 함)

▼ 힌트

1. 전체 키워드를 담을 수 있는 빈 소쿠리를 만듭니다.
2. 각 식당들의 키워드를 수집해서 빈 소쿠리에 담아줍니다. (힌트: 리스트의 `extend` 를 활용)
3. 전체 키워드가 다 수집되면 `set`로 변환해줍니다. (힌트: `set()` 함수 사용)

▼ 코드 예시

```

# 풀이 전략
# 1. 전체 키워드를 채집할 수 있는 빈 소쿠리를
# 만듭니다.
# 2. 각 식당들의 키워드를 수집해서 빈 소쿠리에
# 담아줍니다. (힌트: 리스트의 extend를 활용)
# 3. 전체 키워드가 다 수집되면 set로 변환해줍니
# 다. (힌트: `set()` 함수 사용)

```

```

all_keywords = []

for restaurant in restaurants:
    keywords = restaurant.get("keywords")
    all_keywords.extend(keywords)

all_keywords = set(all_keywords)
print(all_keywords)

```

5. 각 식당의 **위치(location)**를 기준으로, 현재 강의장 위치에서 **가장 멀리 있는 식당의 거리**를 찾아보세요.

참고로 강의장의 좌표는 **(37.558497, 126.926459)** 입니다.

▼ 거리 구하기 공식

$$\sqrt{((\text{식당과 강의장의 x좌표 차이})^2 + (\text{식당과 강의장의 y좌표 차이})^2)}$$

▼ 코드 예시

챔피언 전략을 쓴다!

```

from math import sqrt

# 챔피언 자리를 만듦. 처음에는 무주공산으로 만듦.
max_distance = -10000000
max_distance_name = ""

current_location = (37.558497, 126.926459)

for restaurant in restaurants:
    if "location" in restaurant:
        location = restaurant["location"]
        distance = sqrt((current_location[0] - location[0]) ** 2 + (current_location[1] - location[1]) ** 2)

        if max_distance < distance:
            max_distance = distance
            max_distance_name = restaurant["name"]

```

```
print(max_distance)
print(max_distance_name)
```

▼ 학교 도서관 관리 시스템

다음과 같은 도서관 데이터가 주어졌을 때, 아래 문제들을 해결해보세요:

```
# 도서 정보 (제목, 저자, 출판연도, 장르, 대출 가능 여부)
books = {
    "B001": ("파이썬 프로그래밍", "김코딩", 2021, "컴퓨터",
True),
    "B002": ("미적분의 이해", "이수학", 2019, "수학", False),
    "B003": ("한국사 개론", "박역사", 2020, "역사", True),
    "B004": ("생명과학 입문", "최생물", 2022, "과학", True),
    "B005": ("프로그래밍 언어론", "정컴퓨터", 2021, "컴퓨터", False)
}

# 학생별 대출 기록 (학생ID: [(도서ID, 대출일, 반납일)])
borrowing_history = {
    "S001": [
        ("B001", "2024-01-15", "2024-02-01"),
        ("B003", "2024-02-10", None)
    ],
    "S002": [
        ("B002", "2024-01-20", None)
    ],
    "S003": [
        ("B005", "2024-02-01", None)
    ]
}

# 학생 정보 (이름, 학년, 선호 장르)
students = {
    "S001": ("김학생", 3, ["컴퓨터", "과학"]),
```

```
"S002": ("이학생", 2, ["수학", "과학"]),
"S003": ("박학생", 1, ["컴퓨터", "역사"])
}
```

▼ 기본 문제

1. 현재 대출 가능한 도서의 제목들을 리스트로 반환하는 함수를 작성하세요.
2. 특정 장르의 도서들을 찾아 (도서ID, 제목) 형태의 리스트로 반환하는 함수를 작성하세요.
3. 현재 연체 중인 학생들의 이름을 찾아 반환하는 함수를 작성하세요. (반납일이 None이면 연체로 간주)

▼ 중급 문제

1. 각 장르별 도서의 수를 딕셔너리 형태로 반환하는 함수를 작성하세요.
2. 특정 학생이 자신의 선호 장르의 책들 중 아직 읽지 않은 도서를 추천받는 함수를 작성하세요.
3. 가장 많이 대출된 도서의 ID와 대출 횟수를 튜플로 반환하는 함수를 작성하세요.

▼ 고급 문제

1. 각 학년별로 가장 선호하는 장르를 찾아 딕셔너리로 반환하는 함수를 작성하세요. (동률일 경우 알파벳 순)
2. 특정 기간 동안 가장 활발하게 도서를 대출한 학생을 찾는 함수를 작성하세요. (대출 횟수와 총 대출 기간을 모두 고려)

▼ 힌트

- 딕셔너리의 items(), keys(), values() 메서드를 활용하면 데이터를 순회하기 쉽습니다.
- 날짜 처리는 datetime 모듈을 사용하면 편리합니다.
- 여러 조건을 처리할 때는 리스트 컴프리헨션이나 필터 함수를 활용해보세요.
- 데이터를 그룹화할 때는 defaultdict를 사용하면 코드를 간단히 할 수 있습니다.